

PROJECT REPORT

Alumni Sync

Alumni Management System — A Database-Driven Web Platform for Alumni Engagement

Python (Flask) • MySQL • HTML5 / CSS3

Prepared by

Gnanendhiran V

Institution

Anna University

Category

DBMS-Focused Full-Stack Mini
Project

Table of Contents

1. Abstract.....	3
2. Introduction.....	3
3. Existing System vs. Proposed System.....	3
4. System Requirements.....	4
5. System Architecture.....	4
6. Database Design.....	5
7. Module Description.....	7
8. Implementation Highlights.....	8
9. Testing.....	8
10. Advantages.....	8
11. Limitations & Future Enhancements.....	8
12. Conclusion.....	9
13. References.....	9

1. Abstract

Most colleges and universities lose touch with their alumni soon after graduation because records are kept in scattered spreadsheets, WhatsApp groups, or not maintained at all. Alumni Sync is a full-stack web application that solves this by giving an institution one authenticated platform where alumni and current students can find each other, share job openings, organise and register for events, request references or mentorship, and send direct messages. The system is built with Python and Flask on the backend, MySQL as the relational data store, and HTML5, CSS3 and JavaScript on the frontend. The emphasis of the project is on sound relational database design — the schema is normalised, referential integrity is enforced with foreign keys and unique constraints, and the data layer is isolated behind a single connection module so the rest of the application never talks to MySQL directly. This report documents the problem the system addresses, its architecture, its database design, and the modules that were implemented.

2. Introduction

2.1 Problem Statement

Institutions rarely have a single, reliable channel to stay connected with graduated students. Placement cells struggle to source referrals, students find it hard to reach alumni working in their field of interest, and event organisers have no dedicated way to invite and track alumni participation. The absence of a structured database also means there is no easy way to search alumni by batch, department, company or skill.

2.2 Objectives

- Maintain a single, verified directory of alumni and students behind secure, role-based logins.
- Let alumni post job or internship openings and let students apply directly through the platform.
- Allow event organisers to publish events and track registrations in real time.
- Provide a private messaging channel and a reference/mentorship request workflow between students and alumni.
- Demonstrate relational database design practices — normalisation, keys, views, triggers and indexing — on a realistic, multi-entity domain.

2.3 Scope

The system covers three user roles — Admin, Alumni and Student — and the core workflows of profile management, job posting and application, event management, messaging, reference requests and announcements. Payment processing, video calling and mobile applications are outside the scope of the current version and are listed under future enhancements.

3. Existing System vs. Proposed System

Aspect	Existing Practice	Proposed System
Alumni records	Spreadsheets / paper registers maintained manually	Centralised, normalised MySQL database
Job referrals	Informal WhatsApp / email forwards	Structured job board with tracked

Aspect	Existing Practice	Proposed System
		applications
Event coordination	Circulars and manual RSVP lists	Online event creation with registration tracking
Alumni search	No searchable index by batch/department/skill	Filterable, indexed alumni directory
Access control	No access control; shared documents	Role-based authentication (Admin / Alumni / Student)

4. System Requirements

4.1 Software Requirements

Component	Technology
Programming Language	Python 3.8+
Web Framework	Flask
Database	MySQL 8.0 (InnoDB)
DB Driver	mysql-connector-python
Templating	Jinja2
Frontend	HTML5, CSS3, JavaScript
Authentication	Werkzeug security (password hashing) + session cookies
Config Management	python-dotenv (.env file)
Version Control	Git & GitHub

4.2 Hardware Requirements

Resource	Minimum
Processor	Dual-core 1.8 GHz or higher
RAM	4 GB (8 GB recommended)
Storage	500 MB free space for application and uploads
Network	Internet connection for package installation and deployment

5. System Architecture

The application follows a layered architecture that keeps presentation, application logic and data access independent of one another. The browser talks to Flask routes over HTTP; routes check the session through a lightweight auth layer built on Werkzeug's password-hashing utilities and then read or write data through a single data-access module, `db_connect.py`, which every route calls through one `execute_query()` helper rather than opening MySQL connections directly. A small family of standalone scripts — `init_db.py`, `populate_data.py`, `clean_db.py` and `check_db.py` — sit alongside the application to build the schema, seed sample data, reset the database, and sanity-check it during development.

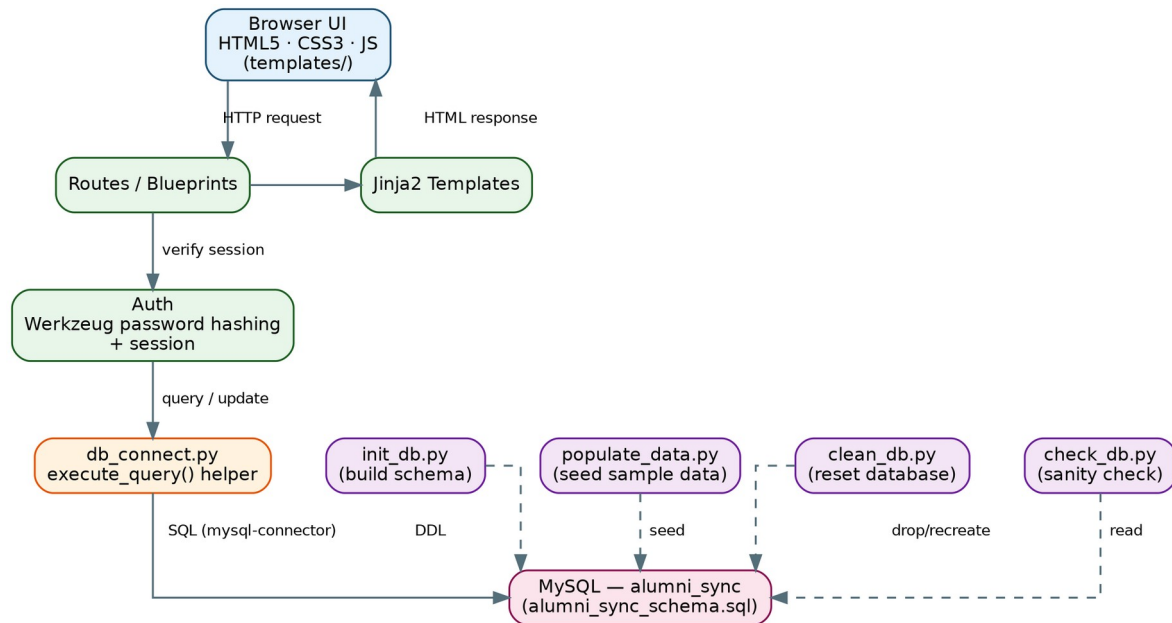


Figure 5.1 — Layered architecture of Alumni Sync

6. Database Design

The relational schema is the core of this project. Every account — alumni, student or admin — has exactly one row in a central users table that stores login credentials and a role reference; role-specific attributes are kept in separate `alumni_profiles` and `student_profiles` tables linked back by a unique foreign key. This 1:1 extension pattern keeps the users table narrow and avoids the large block of nullable columns that would appear if alumni-only and student-only fields were merged into one table. Transactional entities — job postings, applications, events, registrations, reference requests and announcements — are modelled as their own tables connected to users (and to each other) through foreign keys, so nothing has to be duplicated or re-derived on read.

6.1 Entity–Relationship Diagram

The diagram below shows the ten tables that carry the core workflow of the system. Supporting tables that exist in the schema for secondary concerns (audit logs, tokens, media metadata, static "About" page content, etc.) are omitted here to keep the diagram readable — the same 1:1 / 1:N patterns extend consistently to those tables.

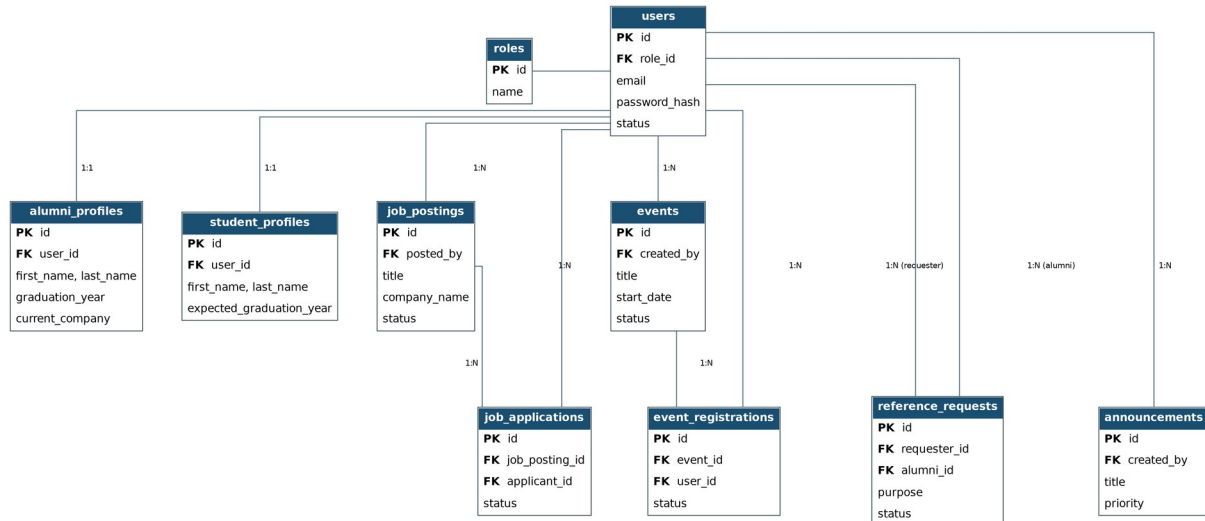


Figure 6.1 — Entity-Relationship diagram of the core schema (key tables)

6.2 Core Tables

Table	Purpose	Key Relationships
roles	Defines the three account types: admin, alumni, guest	Parent of users via role_id
users	Login credentials, role and account status	Parent of nearly every other table via user_id
alumni_profiles	Name, graduation year, company, designation, bio	1:1 with users
student_profiles	Name, degree, expected graduation year, CGPA	1:1 with users
job_postings	Job/internship openings created by alumni	N:1 with users (posted_by)
job_applications	Applications submitted against a posting	N:1 with job_postings and users (applicant_id)
events	Events created by alumni or admin	N:1 with users (created_by)
event_registrations	Tracks who registered for which event	N:1 with events and users
reference_requests	Mentorship/reference requests to alumni	N:1 with users, twice (requester_id, alumni_id)
announcements	Institution-wide notices	N:1 with users (created_by)

6.3 Normalisation and Integrity

- 1NF: every column holds a single atomic value — for example, alumni skills live in their own alumni_skills table instead of a repeating comma-separated field.
- 2NF/3NF: role-specific attributes were split out of users into alumni_profiles / student_profiles, and static reference data (degree names, campus names) lives in its own degrees and campuses tables rather than being repeated on every profile row.
- Referential integrity is enforced throughout with FOREIGN KEY constraints — ON DELETE CASCADE where a child row is meaningless without its parent (e.g. event_registrations, job_applications), and ON DELETE SET NULL where the parent is optional context (e.g. a profile's degree or campus).

- Uniqueness rules prevent duplicate data at the database level: one profile per user (UNIQUE user_id), one application per applicant per job, and one registration per user per event.

6.4 Views and Triggers

A handful of views and triggers keep derived data consistent without pushing that logic into the application code:

- vw_dashboard_stats, vw_open_jobs and vw_upcoming_events pre-join the relevant tables so the admin dashboard and listing pages can read a single view instead of repeating multi-table joins.
- tr_job_application_insert / tr_job_application_delete keep job_postings.applications_count in sync automatically whenever a row is added to or removed from job_applications.
- tr_message_insert updates conversations.last_message_at whenever a new message is sent, so conversation lists can be sorted without a separate query.

7. Module Description

7.1 Authentication & User Management

Handles registration, login and logout across the admin/alumni/guest roles defined in the roles table. Passwords are never stored in plain text — they are hashed with Werkzeug's security helpers before being written to users.password_hash, and a session cookie keeps the user authenticated across requests.

7.2 Alumni & Student Directory

A searchable directory lets students look up alumni by name, company or graduation year, and lets alumni browse student profiles, backed by alumni_profiles and student_profiles.

7.3 Job Board

Alumni post openings through job_postings; students browse and apply through job_applications, with each application's status (pending, shortlisted, interviewed, hired, rejected, ...) tracked so both sides can follow progress.

7.4 Event Management

Alumni or admins create events in the events table; any user can register through event_registrations, and a unique constraint on (event_id, user_id) stops duplicate sign-ups.

7.5 Reference & Mentorship Requests

Students request a reference or mentorship from a specific alumnus through reference_requests; the alumnus can accept, decline or complete the request, and an optional rating/feedback pair is captured once it's done.

7.6 Announcements & Admin Tools

Admins broadcast notices through announcements, and use check_db.py and clean_db.py during development to verify or reset the underlying data without touching the schema file by hand.

8. Implementation Highlights

- Role-based access keeps each route restricted to the roles allowed to use it (e.g. only an authenticated alumnus can create a job posting).
- All queries go through `db_connect.py`'s parameterised `execute_query()` helper, which avoids string-built SQL and the injection risk that comes with it.
- Configuration (DB host, user, password, secret key) is kept out of source control and loaded at runtime from a `.env` file via `python-dotenv`.
- `setup_and_run.bat` automates environment setup end-to-end on Windows — virtual environment creation, dependency install, `.env` scaffolding, optional database (re)initialisation, and finally launching the app.

9. Testing

Test Case	Input	Expected Result	Status
User registration	Valid email & password	Account created with hashed password, status=pending	Pass
Duplicate email signup	Already-registered email	Rejected — unique constraint on users.email	Pass
Invalid login	Wrong password	Access denied, no session created	Pass
Job application	Student applies to an open posting	Row inserted in job_applications	Pass
Duplicate application	Same applicant applies twice	Blocked by unique_application constraint	Pass
Duplicate event registration	Same user registers twice	Blocked by unique_registration constraint	Pass
Database reset	Run <code>clean_db.py</code> then <code>init_db.py</code>	Schema recreated cleanly with no leftover data	Pass

10. Advantages

- Replaces scattered spreadsheets with a single, query-able source of truth.
- Role-based access keeps sensitive contact information visible only to verified users.
- Automatic statistics via triggers/views remove the need for expensive on-demand aggregation.
- Normalised schema keeps storage efficient and avoids update anomalies as the alumni base grows.

11. Limitations & Future Enhancements

- Real-time messaging (WebSockets) could be layered on top of the existing conversations/messages tables for instant delivery.
- A recommendation engine could suggest relevant jobs to students based on skills and degree.

- Payment integration would allow paid alumni-association memberships or event ticketing.
- A dedicated mobile application would extend reach beyond the web interface.

12. Conclusion

Alumni Sync demonstrates how a carefully normalised relational schema, combined with a clean layered architecture, can turn a scattered, manual process into a searchable, auditable platform. Beyond the web features, the project's real emphasis is on database engineering — entity design, keys, constraints and a disciplined data-access layer — applied to a domain with realistic, interconnected entities. The result is a system that is straightforward to extend with new modules without disturbing the existing schema.

13. References

1. Flask Documentation — <https://flask.palletsprojects.com/>
2. MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
3. Werkzeug Security Helpers — <https://werkzeug.palletsprojects.com/en/latest/utils/#module-werkzeug.security>
4. mysql-connector-python Documentation — <https://dev.mysql.com/doc/connector-python/en/>